

```

import os
import random
from dotenv import load_dotenv
from huggingface_hub import login
from datasets import load_dataset, Dataset, DatasetDict
from loaders import ItemLoader
import matplotlib.pyplot as plt
from collections import Counter, defaultdict
import numpy as np
import pickle
from items import Item

load_dotenv(override=True)
os.environ['OPENAI_API_KEY'] = os.getenv('OPENAI_API_KEY', 'your-key-if-not-using-env')
os.environ['ANTHROPIC_API_KEY'] = os.getenv('ANTHROPIC_API_KEY', 'your-key-if-not-using-env')
os.environ['HF_TOKEN'] = os.getenv('HF_TOKEN', 'your-key-if-not-using-env')

hf_token = os.environ['HF_TOKEN']
login(hf_token, add_to_git_credential=True)

%matplotlib inline

```

Hàm/Dòng lệnh	Chức năng
import os	Nhập module os để làm việc với biến môi trường.
import random	Nhập module random để tạo số ngẫu nhiên.
from dotenv import load_dotenv	Nhập load_dotenv để tải biến môi trường từ file .env.
from huggingface_hub import login	Nhập login để đăng nhập vào Hugging Face.
from datasets import load_dataset, Dataset, DatasetDict	Nhập các lớp và hàm để làm việc với tập dữ liệu từ Hugging Face.
from loaders import ItemLoader	Nhập lớp ItemLoader từ module loaders.
import matplotlib.pyplot as plt	Nhập matplotlib.pyplot để vẽ biểu đồ.
from collections import Counter, defaultdict	Nhập Counter và defaultdict để đếm và quản lý dữ liệu dạng từ điển.
import numpy as np	Nhập numpy để xử lý dữ liệu số.
import pickle	Nhập pickle để lưu và tải dữ liệu dưới dạng nhị phân.
from items import Item	Nhập lớp Item từ module items.
load_dotenv(override=True)	Tải các biến môi trường từ file .env, ghi đè nếu đã có sẵn.
os.environ['OPENAI_API_KEY'] = os.getenv('OPENAI_API_KEY', 'your-key-if-not-using-env')	Lấy API key của OpenAI từ biến môi trường.
os.environ['ANTHROPIC_API_KEY'] = os.getenv('ANTHROPIC_API_KEY', 'your-key-if-not-using-env')	Lấy API key của Anthropic từ biến môi trường.

env')	
os.environ['HF_TOKEN'] = os.getenv('HF_TOKEN', 'your- key-if-not-using-env')	Lấy token Hugging Face từ biến môi trường.
hf_token = os.environ['HF_TOKEN']	Lưu token Hugging Face vào biến hf_token.
login(hf_token, add_to_git_credential=True)	Đăng nhập vào Hugging Face bằng token và lưu thông tin vào Git Credential.
%matplotlib inline	Hiển thị biểu đồ trực tiếp trong notebook Jupyter.

<pre> items = ItemLoader("Appliances").load() print(items[1].prompt) dataset_names = ["Automotive", "Electronics", "Office_Products", "Tools_and_Home_Improvement", "Cell_Phones_and_Accessories", "Toys_and_Games", "Appliances", "Musical_Instruments",] items = [] for dataset_name in dataset_names: loader = ItemLoader(dataset_name) items.extend(loader.load()) print(f"A grand total of {len(items):,} items") </pre>	
Hàm/Dòng lệnh	Chức năng
items = ItemLoader("Appliances").load()	Tạo đối tượng ItemLoader cho danh mục "Appliances" và tải dữ liệu sản phẩm.
print(items[1].prompt)	In ra thuộc tính prompt của phần tử thứ hai trong danh sách items.
dataset_names = [...]	Khai báo danh sách các tên bộ dữ liệu.
items = []	Khởi tạo danh sách trống items.
for dataset_name in dataset_names:	Lặp qua tất cả các bộ dữ liệu trong dataset_names.
loader = ItemLoader(dataset_name)	Tạo đối tượng ItemLoader cho mỗi bộ dữ liệu.
items.extend(loader.load())	Tải dữ liệu và mở rộng danh sách items với các sản phẩm từ bộ dữ liệu hiện tại.
print(f'A grand total of {len(items):,} items')	In tổng số lượng sản phẩm trong danh sách items.

```

tokens = [item.token_count for item in items]
plt.figure(figsize=(15, 6))
plt.title(f"Token counts: Avg {sum(tokens)/len(tokens):.1f} and highest {max(tokens):,}\n")
plt.xlabel('Length (tokens)')
plt.ylabel('Count')
plt.hist(tokens, rwidth=0.7, color="skyblue", bins=range(0, 300, 10))
plt.show()

prices = [item.price for item in items]
plt.figure(figsize=(15, 6))
plt.title(f"Prices: Avg {sum(prices)/len(prices):.1f} and highest {max(prices):,}\n")
plt.xlabel('Price ($)')
plt.ylabel('Count')
plt.hist(prices, rwidth=0.7, color="blueviolet", bins=range(0, 1000, 10))
plt.show()

```

Hàm/Dòng lệnh	Chức năng
tokens = [item.token_count for item in items]	Tạo danh sách số lượng token của từng sản phẩm.
plt.figure(figsize=(15, 6))	Tạo một biểu đồ mới với kích thước 15x6 inch.
plt.title(f"Token counts: Avg {sum(tokens)/len(tokens):.1f} and highest {max(tokens):,}\n")	Đặt tiêu đề cho biểu đồ số lượng token, hiển thị giá trị trung bình và cao nhất.
plt.xlabel('Length (tokens)')	Gán nhãn trục X là "Length (tokens)".
plt.ylabel('Count')	Gán nhãn trục Y là "Count".
plt.hist(tokens, rwidth=0.7, color="skyblue", bins=range(0, 300, 10))	Vẽ biểu đồ histogram số lượng token với màu xanh dương nhạt, phạm vi từ 0 đến 300, bước nhảy 10.
plt.show()	Hiển thị biểu đồ số lượng token.
prices = [item.price for item in items]	Tạo danh sách giá của các sản phẩm hợp lệ.
plt.figure(figsize=(15, 6))	Tạo một biểu đồ mới với kích thước 15x6 inch.
plt.title(f"Prices: Avg {sum(prices)/len(prices):.1f} and highest {max(prices):,}\n")	Đặt tiêu đề cho biểu đồ giá sản phẩm, hiển thị giá trị trung bình và cao nhất.
plt.xlabel('Price (\$)')	Gán nhãn trục X là "Price (\$)".
plt.ylabel('Count')	Gán nhãn trục Y là "Count".
plt.hist(prices, rwidth=0.7, color="blueviolet", bins=range(0, 1000, 10))	Vẽ biểu đồ histogram giá sản phẩm với màu tím xanh, phạm vi từ 0 đến 1000, bước nhảy 10.

```

category_counts = Counter()
for item in items:
    category_counts[item.category]+=1

categories = category_counts.keys()
counts = [category_counts[category] for category in categories]

plt.figure(figsize=(15, 6))
plt.bar(categories, counts, color="goldenrod")
plt.title('How many in each category')
plt.xlabel('Categories')
plt.ylabel('Count')

plt.xticks(rotation=30, ha='right')

for i, v in enumerate(counts):
    plt.text(i, v, f"{v:,.}", ha='center', va='bottom')

plt.show()

```

Hàm/Dòng lệnh	Chức năng
category_counts = Counter()	Khởi tạo một đối tượng Counter để đếm số lượng sản phẩm theo danh mục.
for item in items:	Lặp qua từng sản phẩm trong danh sách items.
category_counts[item.category] += 1	Tăng số lượng sản phẩm trong danh mục tương ứng.
categories = category_counts.keys()	Lấy danh sách các danh mục.
counts = [category_counts[category] for category in categories]	Tạo danh sách số lượng sản phẩm cho mỗi danh mục.
plt.figure(figsize=(15, 6))	Tạo một biểu đồ mới với kích thước 15x6 inch.
plt.bar(categories, counts, color="goldenrod")	Vẽ biểu đồ cột với các danh mục và số lượng sản phẩm.
plt.xticks(rotation=30, ha='right')	Quay nhãn trục X 30 độ và căn phải.
for i, v in enumerate(counts):	Lặp qua từng giá trị trong danh sách counts.
plt.text(i, v, f"{v:,.}", ha='center', va='bottom')	Thêm nhãn số lượng lên trên các cột trong biểu đồ.
plt.show()	Hiển thị biểu đồ.

```

slots = defaultdict(list)
for item in items:
    slots[round(item.price)].append(item)

np.random.seed(42)
random.seed(42)
sample = []
for i in range(1, 1000):
    slot = slots[i]
    if i>=240:
        sample.extend(slot)
    elif len(slot) <= 1200:
        sample.extend(slot)
    else:
        weights = np.array([1 if item.category=='Automotive' else 5 for
item in slot])
        weights = weights / np.sum(weights)
        selected_indices = np.random.choice(len(slot), size=1200,
replace=False, p=weights)
        selected = [slot[i] for i in selected_indices]
        sample.extend(selected)

print(f"There are {len(sample):.} items in the sample")

```

Hàm/Dòng lệnh	Chức năng
slots = defaultdict(list)	Khởi tạo một đối tượng defaultdict để lưu trữ các sản phẩm theo giá.
for item in items:	Lặp qua từng sản phẩm trong danh sách items.
slots[round(item.price)].append(item)	Phân loại sản phẩm vào các danh mục theo giá đã làm tròn.
np.random.seed(42)	Đặt giá trị seed cho NumPy để tái tạo kết quả ngẫu nhiên.
random.seed(42)	Đặt giá trị seed cho thư viện random để tái tạo kết quả ngẫu nhiên.
sample = []	Khởi tạo danh sách trống sample để lưu trữ các sản phẩm mẫu.
for i in range(1, 1000):	Lặp qua tất cả các giá từ 1 đến 999.
slot = slots[i]	Lấy danh sách các sản phẩm trong danh mục có giá bằng i.
if i>=240:	Nếu giá của sản phẩm lớn hơn hoặc bằng 240, lấy tất cả sản phẩm.
sample.extend(slot)	Thêm tất cả sản phẩm vào danh sách sample.
elif len(slot) <= 1200:	Nếu số lượng sản phẩm nhỏ hơn hoặc bằng 1200, lấy tất cả sản phẩm.
weights = np.array([1 if item.category=='Automotive' else 5 for item in slot])	Tính trọng số cho từng sản phẩm, với các sản phẩm thuộc danh mục "Automotive" có trọng số bằng 1 và các danh mục khác có trọng số bằng 5.
weights = weights / np.sum(weights)	Chuẩn hóa trọng số.
selected_indices = np.random.choice(len(slot), size=1200, replace=False,	Chọn 1200 sản phẩm từ danh sách với trọng số đã tính.

p=weights)	
selected = [slot[i] for i in selected_indices]	Lấy các sản phẩm đã được chọn dựa trên chỉ số.
sample.extend(selected)	Thêm các sản phẩm đã chọn vào danh sách sample.
print(f'There are {len(sample):,} items in the sample')	In ra số lượng sản phẩm trong mẫu.

<pre>prices = [float(item.price) for item in sample] plt.figure(figsize=(15, 10)) plt.title(f'Avg {sum(prices)/len(prices):.2f} and highest {max(prices):,.2f}\n') plt.xlabel('Price (\$)') plt.ylabel('Count') plt.hist(prices, rwidth=0.7, color="darkblue", bins=range(0, 1000, 10)) plt.show()</pre>	
Hàm/Dòng lệnh	Chức năng
prices = [float(item.price) for item in sample]	Chuyển đổi giá của các sản phẩm trong mẫu sang kiểu dữ liệu float và tạo danh sách prices.
plt.figure(figsize=(15, 10))	Tạo một biểu đồ mới với kích thước 15x10 inch.
plt.title(f'Avg {sum(prices)/len(prices):.2f} and highest {max(prices):,.2f}\n')	Đặt tiêu đề cho biểu đồ, hiển thị giá trị trung bình và cao nhất của giá sản phẩm.
plt.xlabel('Price (\$)')	Gán nhãn trục X là "Price (\$)".
plt.ylabel('Count')	Gán nhãn trục Y là "Count".
plt.hist(prices, rwidth=0.7, color="darkblue", bins=range(0, 1000, 10))	Vẽ biểu đồ histogram giá sản phẩm với màu xanh đậm, phạm vi từ 0 đến 1000, bước nhảy 10.
plt.show()	Hiển thị biểu đồ.

```

category_counts = Counter()
for item in sample:
    category_counts[item.category]+=1

categories = category_counts.keys()
counts = [category_counts[category] for category in categories]

plt.figure(figsize=(15, 6))
plt.bar(categories, counts, color="lightgreen")

plt.title('How many in each category')
plt.xlabel('Categories')
plt.ylabel('Count')

plt.xticks(rotation=30, ha='right')

for i, v in enumerate(counts):
    plt.text(i, v, f"{v:,.}", ha='center', va='bottom')

plt.show()

```

Hàm/Dòng lệnh	Chức năng
category_counts = Counter()	Khởi tạo một đối tượng Counter để đếm số lượng sản phẩm theo danh mục.
for item in sample:	Lặp qua từng sản phẩm trong danh sách sample.
category_counts[item.category] += 1	Tăng số lượng sản phẩm trong danh mục tương ứng.
categories = category_counts.keys()	Lấy danh sách các danh mục.
counts = [category_counts[category] for category in categories]	Tạo danh sách số lượng sản phẩm cho mỗi danh mục.
plt.figure(figsize=(15, 6))	Tạo một biểu đồ mới với kích thước 15x6 inch.
plt.bar(categories, counts, color="lightgreen")	Vẽ biểu đồ cột với các danh mục và số lượng sản phẩm, màu xanh lá.
plt.title('How many in each category')	Đặt tiêu đề cho biểu đồ.
plt.xlabel('Categories')	Gán nhãn trục X là "Categories".
plt.ylabel('Count')	Gán nhãn trục Y là "Count".
plt.xticks(rotation=30, ha='right')	Quay nhãn trục X 30 độ và căn phải.
for i, v in enumerate(counts):	Lặp qua từng giá trị trong danh sách counts.
plt.text(i, v, f"{v:,.}", ha='center', va='bottom')	Thêm nhãn số lượng lên trên các cột trong biểu đồ.
plt.show()	Hiển thị biểu đồ.

```
plt.figure(figsize=(12, 10))
plt.pie(counts, labels=categories, autopct='%1.0f%%', startangle=90)

centre_circle = plt.Circle((0,0), 0.70, fc='white')
fig = plt.gcf()
fig.gca().add_artist(centre_circle)
plt.title('Categories')

plt.axis('equal')

plt.show()
```

Hàm/Dòng lệnh	Chức năng
plt.figure(figsize=(12, 10))	Tạo một hình mới với kích thước chiều rộng 12 inch và chiều cao 10 inch.
plt.pie(counts, labels=categories, autopct='%1.0f%%', startangle=90)	Vẽ biểu đồ tròn với các giá trị từ counts, nhãn từ categories, hiển thị phần trăm với 0 chữ số thập phân, và bắt đầu vẽ từ góc 90 độ.
centre_circle = plt.Circle((0, 0), 0.70, fc='white')	Tạo một hình tròn màu trắng ở trung tâm biểu đồ để tạo hiệu ứng "biểu đồ vòng".
fig = plt.gcf()	Lấy đối tượng hình hiện tại.
fig.gca().add_artist(centre_circle)	Thêm hình tròn vào đối tượng hình hiện tại.
plt.title('Categories')	Thiết lập tiêu đề của biểu đồ là "Categories".
plt.axis('equal')	Đảm bảo tỷ lệ của các trục là bằng nhau, làm cho biểu đồ tròn không bị méo.
plt.show()	Hiển thị biểu đồ.

```
sizes = [len(item.prompt) for item in sample]
prices = [item.price for item in sample]

plt.figure(figsize=(15, 8))
plt.scatter(sizes, prices, s=0.2, color="red")

plt.xlabel('Size')
plt.ylabel('Price')
plt.title('Is there a simple correlation?')

plt.show()

def report(item):
    prompt = item.prompt
    tokens = Item.tokenizer.encode(item.prompt)
    print(prompt)
    print(tokens[-10:])
    print(Item.tokenizer.batch_decode(tokens[-10:]))

report(sample[398000])
```

Hàm/Dòng lệnh	Chức năng
sizes = [len(item.prompt) for item in sample]	Tạo danh sách sizes chứa độ dài của mỗi chuỗi prompt từ từng phần tử trong sample.
prices = [item.price for item in sample]	Tạo danh sách prices chứa giá trị price từ từng phần tử trong sample.

sample]	phần tử trong sample.
plt.figure(figsize=(15, 8))	Tạo một hình mới với kích thước chiều rộng 15 inch và chiều cao 8 inch.
plt.scatter(sizes, prices, s=0.2, color="red")	Vẽ biểu đồ phân tán với sizes là trục x, prices là trục y, điểm có kích thước 0.2 và màu đỏ.
plt.xlabel('Size')	Thiết lập nhãn trục x là "Size".
plt.ylabel('Price')	Thiết lập nhãn trục y là "Price".
plt.title('Is there a simple correlation?')	Thiết lập tiêu đề cho biểu đồ là "Is there a simple correlation?".
plt.show()	Hiển thị biểu đồ phân tán.
def report(item):	Định nghĩa hàm report để xử lý và in thông tin liên quan đến item.
prompt = item.prompt	Lấy giá trị prompt từ đối tượng item.
tokens = Item.tokenizer.encode(item.prompt)	Mã hóa chuỗi prompt của item thành các token bằng bộ mã hóa Item.tokenizer.
print(prompt)	In giá trị prompt.
print(tokens[-10:])	In 10 token cuối cùng của danh sách tokens.
print(Item.tokenizer.batch_decode(tokens[-10:]))	Giải mã và in ra các token cuối cùng vừa in ra trước đó.
report(sample[398000])	Gọi hàm report với phần tử sample[398000].

<pre> random.seed(42) random.shuffle(sample) train = sample[:400_000] test = sample[400_000:402_000] print(f"Divided into a training set of {len(train):,} items and test set of {len(test):,} items") print(train[0].prompt) print(test[0].test_prompt()) </pre>	
Hàm/Dòng lệnh	Chức năng
random.seed(42)	Đặt giá trị seed cho thư viện random để đảm bảo tính ngẫu nhiên có thể tái tạo.
random.shuffle(sample)	Xáo trộn ngẫu nhiên các phần tử trong danh sách sample.
train = sample[:400_000]	Chia danh sách sample thành tập huấn luyện train chứa 400,000 phần tử đầu tiên.
test = sample[400_000:402_000]	Chia danh sách sample thành tập kiểm tra test chứa 2,000 phần tử tiếp theo sau 400,000.
print(f"Divided into a training set of {len(train):,} items and test set of {len(test):,} items")	In ra thông tin về số lượng phần tử trong tập huấn luyện và tập kiểm tra.
print(train[0].prompt)	In ra giá trị prompt của phần tử đầu tiên trong tập huấn luyện train.
print(test[0].test_prompt())	In ra giá trị test_prompt() của phần tử đầu tiên trong tập kiểm tra test.

```

prices = [float(item.price) for item in test[:250]]
plt.figure(figsize=(15, 6))
plt.title(f"Avg {sum(prices)/len(prices):.2f} and highest
{max(prices):.2f}\n")
plt.xlabel('Price ($)')
plt.ylabel('Count')
plt.hist(prices, rwidth=0.7, color="darkblue", bins=range(0, 1000, 10))
plt.show()

```

Hàm/Dòng lệnh	Chức năng
prices = [float(item.price) for item in test[:250]]	Tạo danh sách prices chứa giá trị price từ các phần tử trong test (dưới 250 phần tử), chuyển đổi sang kiểu float.
plt.figure(figsize=(15, 6))	Tạo một hình mới với kích thước chiều rộng 15 inch và chiều cao 6 inch.
plt.title(f"Avg {sum(prices)/len(prices):.2f} and {max(prices):.2f}\n")	Thiết lập tiêu đề cho biểu đồ với giá trị trung bình và giá trị cao nhất của prices.
plt.xlabel('Price (\$)')	Thiết lập nhãn trục x là "Price (\$)".
plt.ylabel('Count')	Thiết lập nhãn trục y là "Count".
plt.hist(prices, rwidth=0.7, color="darkblue", bins=range(0, 1000, 10))	Vẽ biểu đồ histogram cho prices với độ rộng của các cột là 0.7, màu sắc là "darkblue", và các phân đoạn có kích thước 10 từ 0 đến 1000.
plt.show()	Hiển thị biểu đồ histogram.

```

train_prompts = [item.prompt for item in train]
train_prices = [item.price for item in train]
test_prompts = [item.test_prompt() for item in test]
test_prices = [item.price for item in test]

train_dataset = Dataset.from_dict({"text": train_prompts, "price":
train_prices})
test_dataset = Dataset.from_dict({"text": test_prompts, "price":
test_prices})
dataset = DatasetDict({
    "train": train_dataset,
    "test": test_dataset
})

with open('train.pkl', 'wb') as file:
    pickle.dump(train, file)

with open('test.pkl', 'wb') as file:
    pickle.dump(test, file)

```

Hàm/Dòng lệnh	Chức năng
train_prompts = [item.prompt for item in train]	Tạo danh sách train_prompts chứa giá trị prompt từ các phần tử trong tập huấn luyện train.
train_prices = [item.price for item in train]	Tạo danh sách train_prices chứa giá trị price từ các phần tử trong tập huấn luyện train.

<code>test_prompts = [item.test_prompt() for item in test]</code>	Tạo danh sách <code>test_prompts</code> chứa giá trị <code>test_prompt()</code> từ các phần tử trong tập kiểm tra <code>test</code> .
<code>test_prices = [item.price for item in test]</code>	Tạo danh sách <code>test_prices</code> chứa giá trị <code>price</code> từ các phần tử trong tập kiểm tra <code>test</code> .
<code>train_dataset = Dataset.from_dict({"text": train_prompts, "price": train_prices})</code>	Tạo đối tượng <code>train_dataset</code> từ <code>train_prompts</code> và <code>train_prices</code> dưới dạng một từ điển.
<code>test_dataset = Dataset.from_dict({"text": test_prompts, "price": test_prices})</code>	Tạo đối tượng <code>test_dataset</code> từ <code>test_prompts</code> và <code>test_prices</code> dưới dạng một từ điển.
<code>dataset = DatasetDict({"train": train_dataset, "test": test_dataset})</code>	Tạo đối tượng <code>dataset</code> dưới dạng một <code>DatasetDict</code> bao gồm cả <code>train_dataset</code> và <code>test_dataset</code> .
<code>with open('train.pkl', 'wb') as file:</code>	Mở tệp <code>train.pkl</code> để ghi dữ liệu bằng chế độ nhị phân.
<code>pickle.dump(train, file)</code>	Dùng <code>pickle</code> để ghi đối tượng <code>train</code> vào tệp <code>train.pkl</code> .
<code>with open('test.pkl', 'wb') as file:</code>	Mở tệp <code>test.pkl</code> để ghi dữ liệu bằng chế độ nhị phân.
<code>pickle.dump(test, file)</code>	Dùng <code>pickle</code> để ghi đối tượng <code>test</code> vào tệp <code>test.pkl</code> .